Week 01: two kinds of work

Programming task

Session 2: three short Python tasks.
Predict, change and repair.
Session 3: separate application.

Research task

After class: Research 1.
Your goal, a real system and AI.
Discuss in Week 2 · proposed 5%.

Submit to submit.progmis@gmail.com.
Deadlines and file names will be announced.

Use this guide with the same Week notebook; do not create a second set of weekly tasks.

Programming practice

Values, types, assignment and execution order
Predict → run → change → repair → explain

Three core tasks: understand the Python

Predict values

Read count and "count". Predict the type and product. Run.

Change and observe

Change count after total. Explain the old result; recalculate.

Repair a type bug

"4" * 3 repeats text. Make the result numeric; explain why.

Open Week_01.ipynb → Session 2. Keep a prediction, result and explanation for each task.

Core 1–2: predict, then change one thing

```
count = 4
unit = 3
total = count * unit
print(count)
print("count")
print(type(count).__name__)
print(total)
```

1. Predict: 4, literal text, type and numeric product.
2. Change count after total; observe the stored result.
3. Recalculate before printing; explain the new result.

Use the complete Core 1 and Core 2 starters in the notebook; explain each changed result.

Core 3: find and repair one mistake

```
count = "4"  # Deliberate type bug for a numeric calculation.
unit = 3
total = count * unit
print(total)
# Repair the type, then retest count 4 and count 2.
```

1. Predict the actual output before changing anything.
2. Repair the input type; test counts 4 and 2.
3. Explain why text repetition differs from multiplication.

Edit the matching Core 3 cell. Keep the wrong result, your repair and a retest.

Done: explain the concept, then save

Step	What to do / keep
Predict	Keep all four outputs and explain name / literal / type.
Change	Keep old and recalculated totals; identify the assignment.
Repair	Show numeric results 12 and 6; explain the type change.
Optional	Run the same code from a saved .py file.
Save	Keep code and three records in Week_01.ipynb.

After practice, explain types and why changing an input does not recalculate total.

Project application

Continue in the same notebook after the Python practice.

Apply the idea: one-batch planning note

Step	What to do / keep
Open	Project studio → Independent lab: one batch planning note.
Edit	Bread-pack starter: calculate and label planned_portions.
Run / check	2 portions/container; counts 1, 3, 0 → totals 2, 6, 0.
Write / save	One clearer label, one question, a finding and revision.

This is the existing project task. Keep its results separate from your Python practice.

Research a real information system

What to research

Choose a real system, such as a library loan system. Find out who uses it, what information it keeps and what it helps them do.

Possible directions

Connect it to one skill you want to learn. Consider what an engineer still needs to check when AI helps write code.

A short summary example

A library records loans and due dates. I want to learn how to check dates. Even if AI writes the rule, an engineer must test it.

Discuss in Week 2 · about 15–20 minutes · summarize in your own words and cite your sources.